

Dilema do Agente Duplo

- **Limite de tempo:** 12 minutos.
- **Armazenamento:** 5 GB
- **Ambiente:** uma GPU (≈ 16 GB VRAM), sem internet
- **Tamanho da solução:** `solution.ipynb` ≤ 1 MB
- **Pontuação da baseline:** 0

No centro nacional de IA em Astana, dois modelos computacionais — Modelo R (uma ResNet-18) e Modelo V (uma ViT-Tiny) — estão analisando fotos. Neste momento, ambos os modelos estão fazendo um trabalho perfeito, alcançando 100% de acurácia e concordando em todas as imagens. Para testar quão diferentes seus "cérebros" inteligentes realmente são, o cientista-chefe propõe um desafio: fazer alterações minúsculas, quase invisíveis, nos pixels de cada foto para que o Modelo R e o Modelo V discordem completamente.



1. Tarefa

Dois classificadores de imagens pré-treinados analisam a mesma imagem. Nas imagens fornecidas nesta tarefa, ambos os classificadores apresentam 100% de acurácia.

- **Modelo R:** `torchvision.models.resnet18` (uma CNN, ResNet18).
- **Modelo V:** `timm's vit_tiny_patch16_224` (um Transformer, ViT-Tiny).

Sua tarefa é criar uma pequena alteração ("perturbação") para cada imagem de modo que os dois modelos discordem. Para cada imagem, você deve criar **duas perturbações diferentes**:

- **Tipo A:** após adicioná-la, o Modelo R ainda classifica a imagem corretamente, mas o Modelo V a classifica incorretamente.
- **Tipo B:** após adicioná-la, o Modelo V ainda classifica a imagem corretamente, mas o Modelo R a classifica incorretamente.

Cada perturbação deve ser *pequena* o suficiente para ser difícil de perceber. Perturbações menores recebem pontuações maiores (consulte a Seção 5). A perturbação é aplicada diretamente à imagem original no nível dos pixels.

2. Dados públicos

Um conjunto de imagens é fornecido com a tarefa, organizado em duas divisões — `train` (100 imagens) e `test_public` (100 imagens) — cada uma com imagens de resoluções variadas. Todas as imagens pertencem às 1000 classes do ImageNet-1K, e tanto o Modelo R quanto o Modelo V alcançam 100% de acurácia em ambas as divisões.

Os seguintes arquivos são fornecidos:

```
train/images/*.png      # 100 images in PNG format
train/labels.json       # maps each image index to its correct class
test_public/images/*.png # 100 images in PNG format
test_public/labels.json  # maps each image index to its correct class
```

Durante a avaliação, sua pasta `dataset/test_public/` é substituída de forma transparente por dois conjuntos ocultos de imagens (`test_leaderboard_a` e `test_leaderboard_b`) para a pontuação oficial. Cada um deles contém **100 imagens** em formato PNG e um arquivo de rótulos.

Observação: Para esta tarefa, os rótulos nos datasets de teste estão acessíveis.

3. Formato de saída

Para cada imagem, você deve produzir dois arquivos:

```
{index}_a.pt  # Type A perturbation
{index}_b.pt  # Type B perturbation
```

- `{index}` (0, 1, 2, ...), corresponde ao nome da imagem nos datasets.
- Cada arquivo é um único tensor salvo com `torch.save`. Seu formato deve ser $3 \times H \times W$, em que H e W correspondem à resolução **original** dessa imagem (não 224×224).

- O código deve produzir apenas um arquivo ZIP, `submission.zip`. Coloque todos os arquivos `.pt` no nível superior do arquivo ZIP, sem pasta contêiner nem subdiretórios.

```
submission.zip
├── 0_a.pt
├── 0_b.pt
├── 1_a.pt
└── ...
```

O notebook emitirá um alerta se houver algum problema com o formato de saída.

4. Restrições

- **Modelos:** Você deve usar `torchvision.models.resnet18(pretrained=True)` e `timm.create_model('vit_tiny_patch16_224', pretrained=True)`. Nenhum outro modelo pré-treinado é permitido.
- **Pipeline de transformações (aplicado na avaliação):** `Resize(256) → CenterCrop(224) → Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`. See Section 3 of `baseline.ipynb` para obter detalhes.
- **Resolução da perturbação:** Deve corresponder à resolução **original** da imagem bruta (não 224×224). O tensor é adicionado à imagem bruta *antes* do pipeline de transformações.
- **Formato de saída:** apenas arquivos `.pt` — nenhum PNG/JPG. Os tensores são adicionados à imagem bruta, e os valores dos pixels são limitados a `[0, 1]` antes do pré-processamento.
- **Nomenclatura dos arquivos:** Listagem plana, no formato estrito `{index}_a.pt` / `{index}_b.pt`. Nenhum subdiretório dentro do zip.
- **Bibliotecas:** `torch`, `torchvision`, `timm`.

5. Pontuação

A pontuação final é calculada da seguinte forma. Seja M o número de imagens na divisão, $Score_A$ o número de perturbações Tipo A bem-sucedidas e $Score_B$ o número de perturbações Tipo B bem-sucedidas:

$$S_{final} = \frac{Score_A + Score_B}{2M} \times PF$$

PF é uma função projetada para penalizar perturbações com norma alta e para ser muito sensível próximo ao teto de desempenho. Ela é limitada ao intervalo de 0.5 a 1. A implementação completa pode ser vista na Seção 8 de `solution.ipynb`.

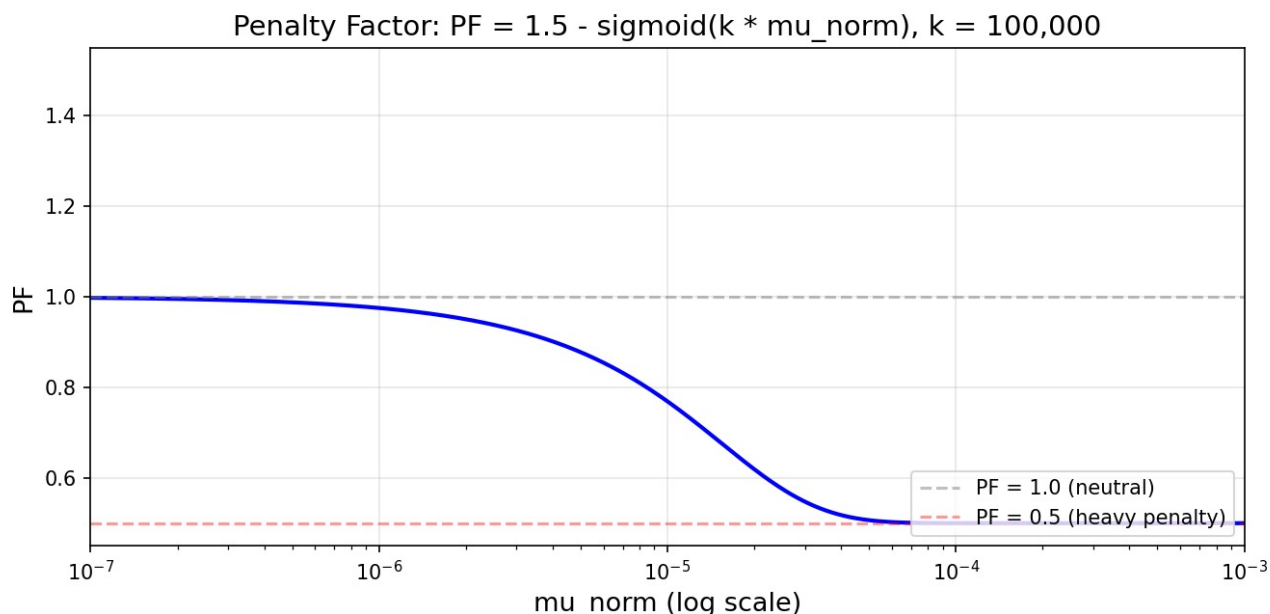


Figura: A curva da função de penalidade.

6. Verificação da Submissão

Há verificações no notebook que alertam você caso haja problemas de formatação, na Seção 7 do notebook `solution.ipynb`.

7. Testes locais

`solution.ipynb` contém um exemplo completo e funcional. Ele carrega os dados públicos, ambos os modelos e o avaliador oficial, e grava um arquivo ZIP de submissão. Leia-o antes de começar.

8. Como submeter

- Salve suas alterações em `solution.ipynb`.
- Abra a aba Git na barra lateral esquerda do JupyterLab.
- Dê **Stage** em `solution.ipynb` (o ícone + ao lado dele).
- Insira uma mensagem de commit e clique em **Commit**.
- Clique no ícone de nuvem com uma seta para cima para fazer push.
- Retorne a esta página da Competição e clique em **Submit**.

Submeta exatamente um arquivo, chamado `solution.ipynb`.